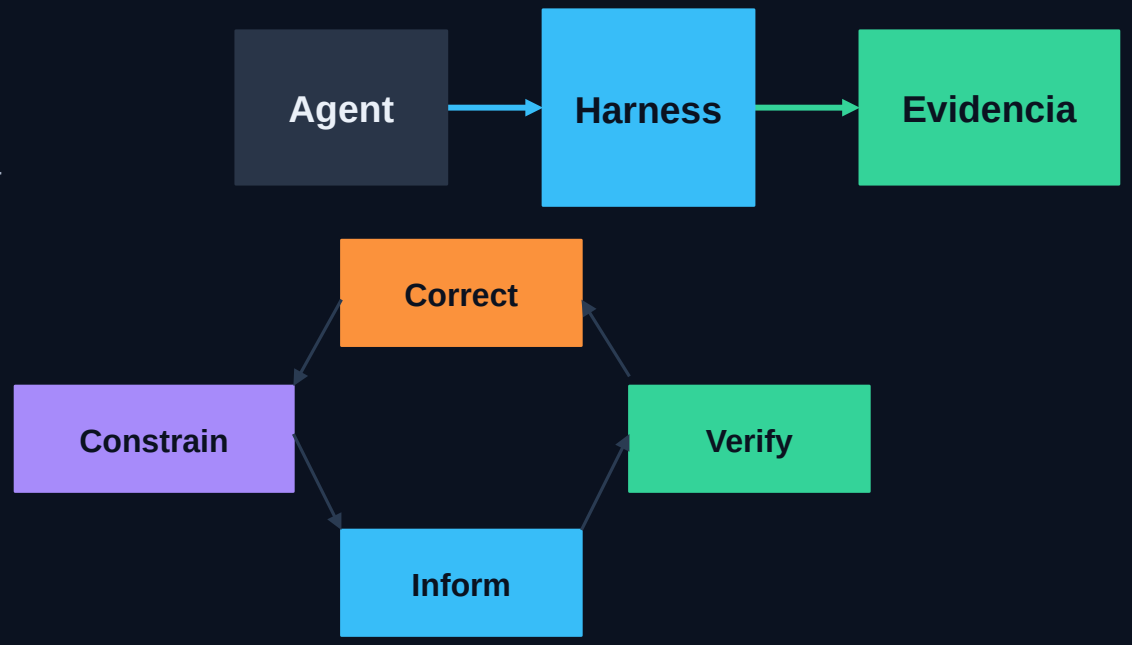
Harness Engineering para Copilot Agents

Como organizar agents, contexto, validacao e evidencias para workflows de IA em software corporativo.



Mapa da apresentacao

O conteudo foi reorganizado em blocos tecnicos, mantendo os assuntos do documento original.

1. **Harness Engineering**
   Controle, contexto, validacao e correcao
2. **Agents e orquestracao**
   Granularidade, routing e especialistas
3. **Context Engineering**
   Camadas, low-token, reset e handoff
4. **Patterns agenticos**
   Chaining, routing, workers, parallel e evaluator
5. **Validacao e riscos**
   DoD, poor self-evaluation, Ralph loop e entropia

Controlar

Delegar

Comprimir

Resetar

Evidenciar

Tese central

Agentes de IA precisam ser tratados como trabalhadores controlados por processo, nao apenas como modelos com prompt.

Risco: o agente pode parecer correto sem ter validado o resultado.

Resultado desejado: o agente so conclui quando existe evidencia verificavel.

Harness Engineering: quatro capacidades

O sistema ao redor do agente deve restringir, informar, verificar e corrigir.

Principios: verification beats generation; trace everything; detect and break loops; reasoning estrategico; onboarding como novo integrante.

Granularidade ideal de Agents

Crie um agent quando houver responsabilidade operacional, contrato de saida e permissao propria.

Nao criar por tecnologia

Nao criar por tela

Nao criar por camada

→ **Agent separado quando >= 3 criterios**

Contrato proprio
Artefato diferente

Ferramentas proprias
Permissoes diferentes

Contexto proprio
Instrucoes especificas

Recorrencia
Tarefa frequente

Validacao propria
Build, lint, review

Se nao atender, prefira: custom instruction, skill, prompt file ou etapa interna do orchestrator.

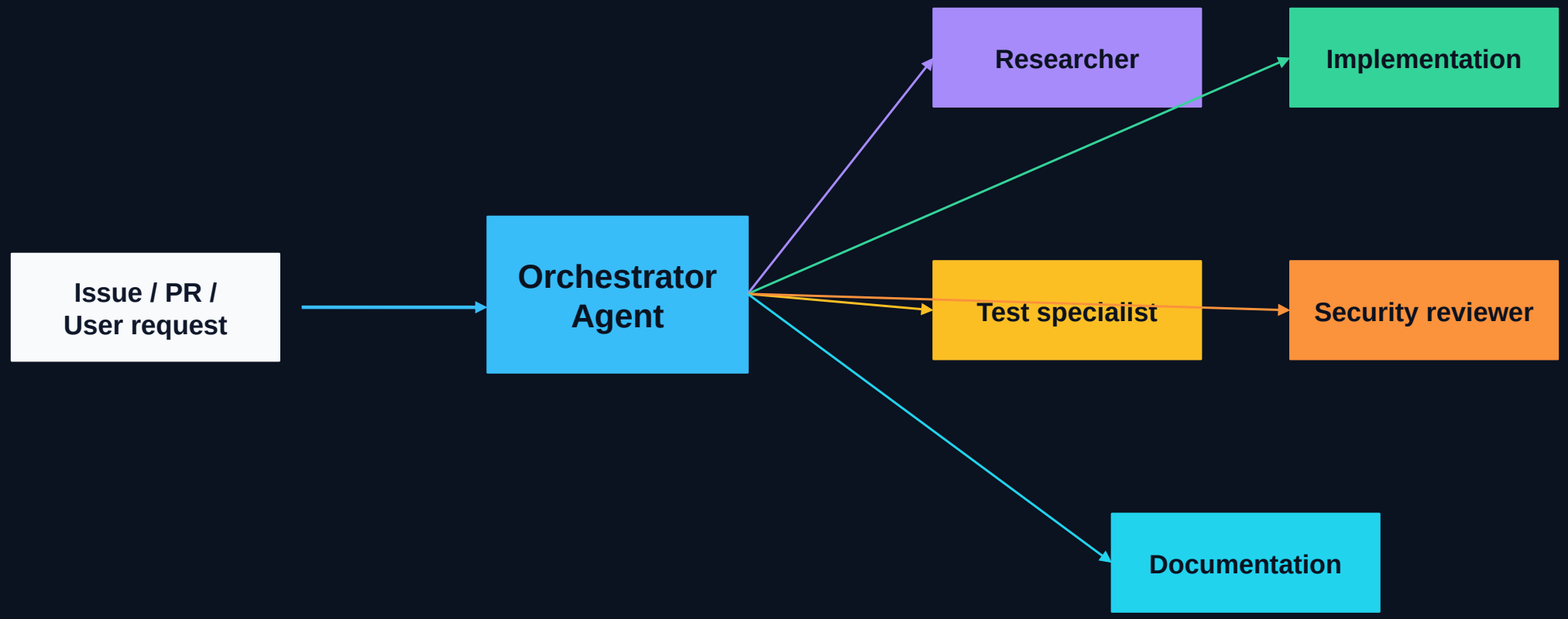
Agent, Skill, Instruction ou MCP?

Escolha o mecanismo pelo tipo de necessidade, nao pelo entusiasmo com agentes.

Regra pratica: so transforme em Custom Agent quando houver comportamento, contexto, ferramentas e validacao proprios.

Arquitetura: Orchestrator + especialistas

O Orchestrator interpreta, roteia, empacota contexto minimo e consolida evidencias.



O Orchestrator nao faz tudo: ele decide o proximo passo com base em contexto, status e evidencias.

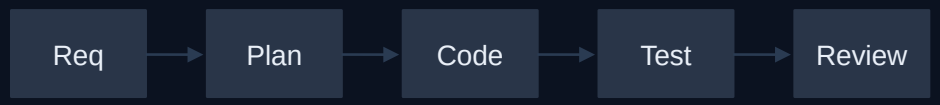
SubAgents devem ter escopo, ferramentas e saida resumida.

Modos de workflow

Escolha o modo conforme dependencia, independencia ou transferencia de responsabilidade.

Sequencial

Requisito -> plano -> implementacao -> teste -> review

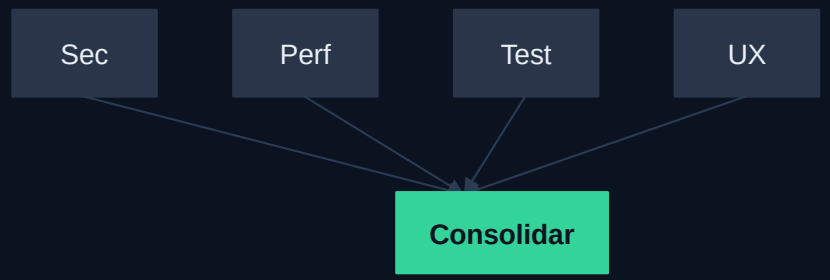


Handoff

Outro agent assume a responsabilidade principal

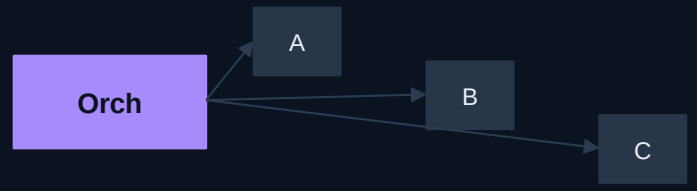
Concorrente

Seguranca, performance, qualidade e UX em paralelo



Manager

Um agent central coordena e consolida tudo



Context Engineering

Curar o contexto certo e evitar tanto a falta quanto o excesso de informacao.

Context Anxiety

Pouco contexto = alucinacao

Contexto demais = ruído e perda de foco

Retrieval

Reranking

Compression

Packet

SubAgent recebe apenas o necessario

Tecnicas: sumarizacao, filtragem de ruido, chunking semantico, semantic trimming, context delta e manifest.

Regra de ouro: passe referencias, nao conteudo; passe decisoes, nao conversas; passe diffs, nao arquivos inteiros.

Camadas de contexto

Cada tipo de contexto deve morar no lugar certo e entrar no prompt apenas quando necessario.

Global	**.github/copilot-instructions.md**	Stack, comandos e padroes gerais
Area	**.github/instructions/*.md**	Frontend, backend, testes, seguranca
Agent	**.github/agents/*.md**	Papel, ferramentas e limites
Sob demanda	**ADRs, OpenAPI, README, repo**	Referenciar; nao colar tudo
Tarefa	**Issue, PR, plano**	Objetivo, aceite e status
Externo	**MCP / Spaces**	Jira, GitHub, design docs

Quanto mais especifico, menos deve entrar automaticamente.

Estrutura recomendada no repositorio

Organizar arquivos de contexto transforma o repo em memoria compartilhada para humanos e agentes.

repo/

.github/copilot-instructions.md	**regras globais**
.github/instructions/*.md	**regras por area**
.github/agents/*.md	**papel e contrato dos agents**
.github/skills/*/SKILL.md	**procedimentos especificos**
docs/architecture.md	**arquitetura atual**
docs/context-map.md	**estado do projeto**
docs/adr/*.md	**decisoes e trade-offs**
docs/api/openapi.yaml	**contrato externo**
docs/design-system.md	**padroes de UX/UI**
CHANGELOG.md	**historico de mudancas**

Antes de implementar, o agente deve ler context-map, instructions, instrucoes da area, ADRs, arquivos recentes e contratos impactados.

Context Impact Checklist
- arquitetura mudou?
- API mudou?
- ADR necessario?
- instructions mudaram?
- context-map atualizado?

Workflow Low-Token

Para tarefas longas, use filesystem, deltas e referencias em vez de historico de chat.

State files	Context compaction	MCP / tools	Context delta	Manifest
.copilot/task_state.json ou .plan.md	Resumo executivo sem explicacoes longas	Buscar conteudo just-in-time	Retornar apenas o que mudou	Mapa de decisoes, riscos e arquivos

Politica de limite sugerida

Context Packet	800 tokens
Research Summary	1000
Implementation Plan	1500
Context Delta	500
Final Agent Report	700

Use arquivos como memoria compartilhada, nao o chat history.

Context Reset com Structured Handoff

Troca de turno: limpar a janela de contexto e passar apenas estado essencial.

O que entra no handoff:
- status atual
- feito / pendente
- arquivos alterados
- decisoes tomadas
- validacoes
- riscos
- proximo passo

O que sai do contexto:
- logs antigos
- tentativas falhas
- mensagens irrelevantes
- ruido de ferramentas
- conversas longas

Fonte de verdade: handoff estruturado + filesystem + Git + testes.

Cinco patterns de Agentic Workflows

Cada pattern resolve uma classe diferente de problema em fluxos com LLMs.

Prompt Chaining	Routing	Orchestrator Workers	Parallelization	Evaluator Optimizer
sequencia fixa	classificar e encaminhar	decompor dinamicamente	executar em paralelo	avaliar e melhorar

Uso recomendado no Copilot

- Chaining como fluxo base previsivel
- Routing para escolher o agent certo
- Orchestrator-Workers para tarefas complexas e multi-arquivos
- Parallelization para reviews independentes
- Evaluator-Optimizer para correcao controlada

O criterio de escolha e a natureza da tarefa: sequencial, categorizavel, dinamica, independente ou iterativa.

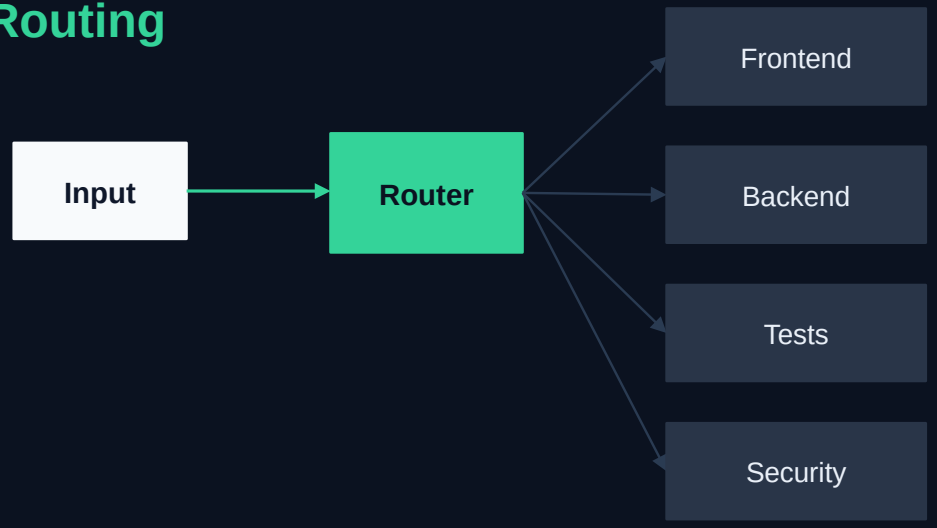
Prompt Chaining e Routing

Dois patterns basicos: decompor passos fixos e direcionar para o especialista correto.

Prompt Chaining

Use quando a tarefa tem etapas claras e dependentes.
Evite para tarefas simples.

Routing

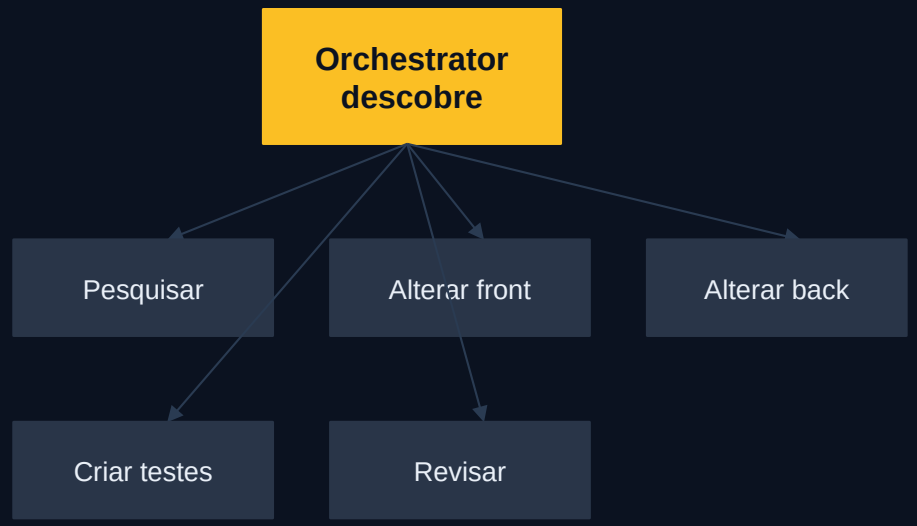


Use quando ha categorias distintas. Cuidado: roteamento ambiguo manda a tarefa ao agent errado.

Orchestrator-Workers x Parallelization

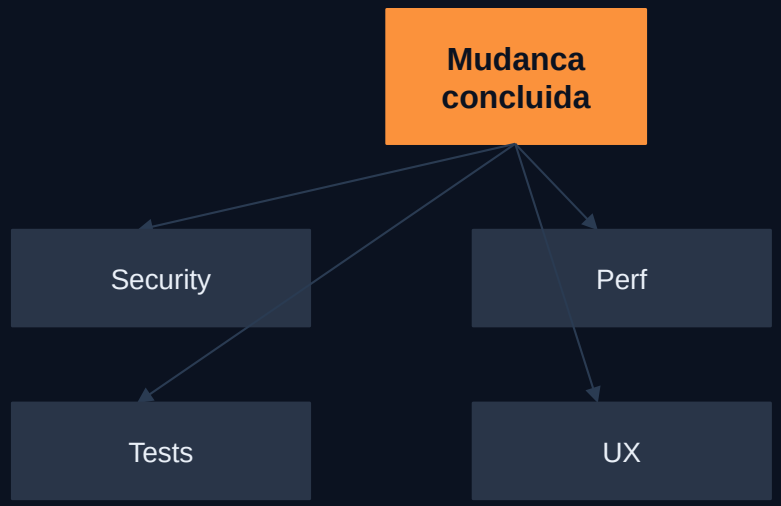
A diferenca principal e se as subtarefas ja sao conhecidas antes.

Orchestrator-Workers



Use quando a tarefa e complexa e o numero de passos depende do input.

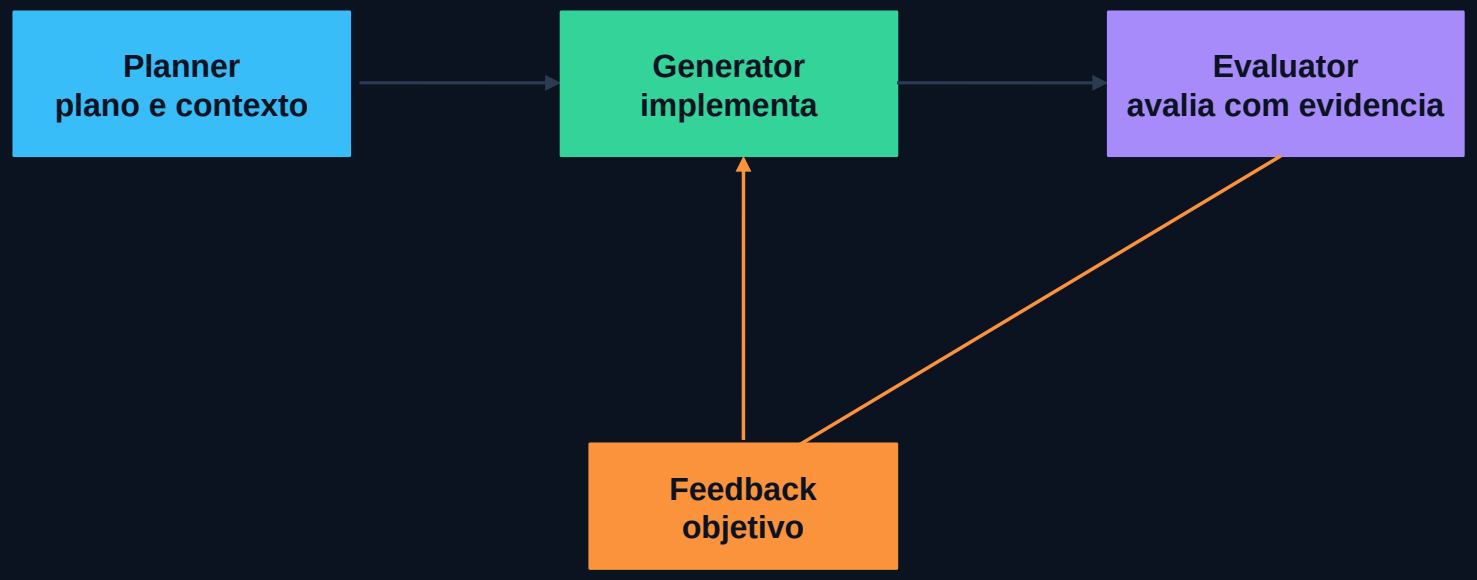
Parallelization



Use quando as analises sao independentes ou quando voting aumenta confianca.

Evaluator-Optimizer e separacao de papeis

O agente que gera a solucao nao deve ser a unica autoridade sobre a conclusao.



Criterios de parada:
- build passa
- testes passam
- lint passa
- aceite verificado
- escopo preservado

Evaluator tambem precisa de ferramentas: testes, lint, diff, contrato de API, screenshots ou validacao E2E.

Quatro falhas comuns em LLMs agentes

O Harness existe para impedir que esses problemas virem falsa produtividade.

One-shots everything	Runs out of context	Leave work half-finished	Declares done early
Tenta resolver tudo em uma tacada	Perde foco ou restricoes antigas	Deixa a branch inconsistente	Confunde codigo com feature pronta

Contramedidas

- features pequenas
- context packets
- clean handoff
- definition of done com evidencias
- test agent + reviewer separado

Regra: codigo escrito nao e feature pronta. Pronto exige evidencia.

Agent Poor Self Evaluation

O agente avalia mal o proprio trabalho quando julga pelo transcript, nao pelo outcome.

Sintomas

- declara done cedo
- ignora edge cases
- aceita evidencia fraca
- confunde build com funcionamento
- nao percebe regressao

Correcao

- test agent valida criterios
- reviewer verifica escopo e riscos
- orchestrator decide status final
- implementation agent apenas relata o que fez

Definition of Done para Agents

O agente so pode declarar conclusao quando cada criterio tiver status e evidencia.

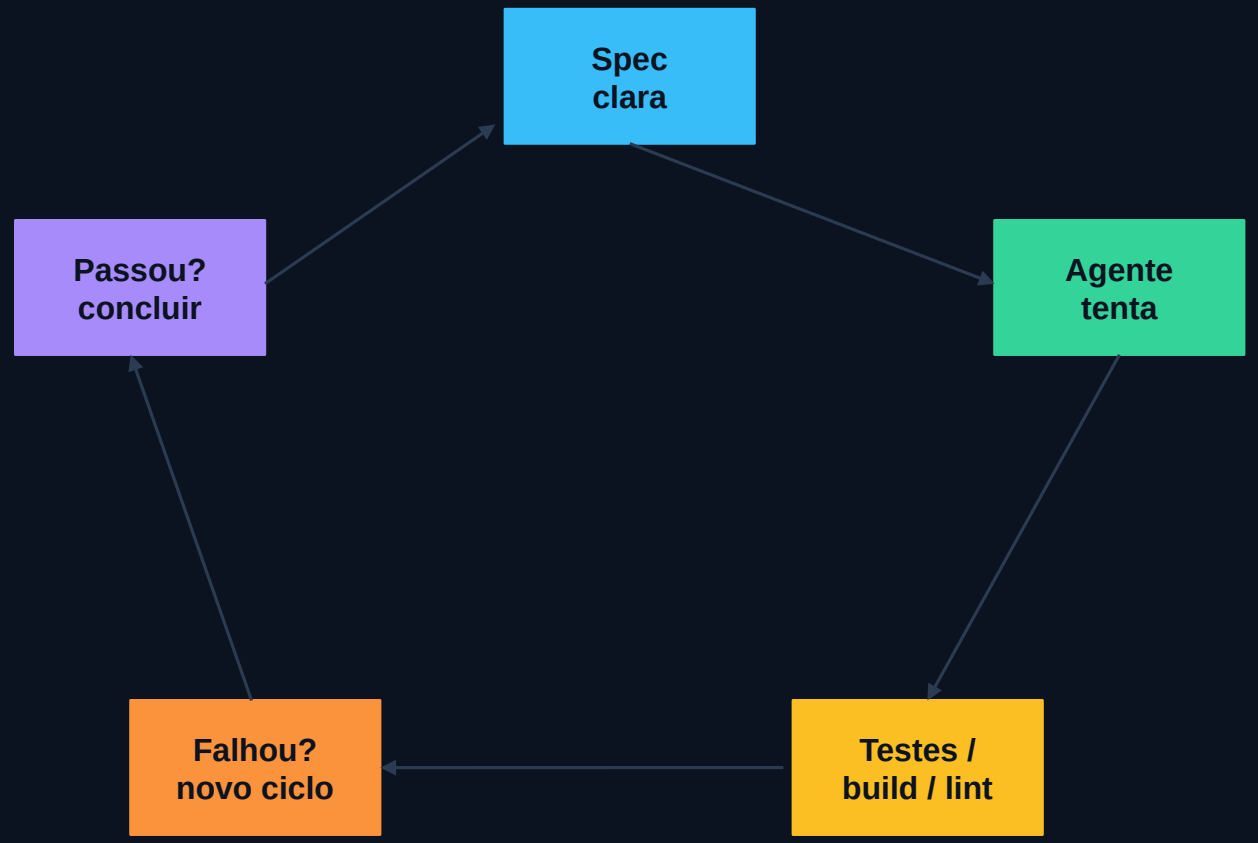
Criterio	Status	Evidencia
Acceptance criteria verificados	Passed	Checklist item a item
Build executado	Passed	comando + resultado
Testes relevantes	Passed/Failed	suite e falhas
Lint/typecheck	Passed/Not run	motivo se nao rodou
Riscos e gaps	Declarados	lista explicita
Itens nao validados	Not validated	nunca esconder incerteza

Nunca trate sua propria confianca como evidencia.

Ralph Wiggum Loop

Persistencia controlada: repetir ciclos curtos ate passar por criterios objetivos ou atingir limite.

Cuidados:
- task pequena
- testes rapidos
- limite de iteracoes
- criterio objetivo



A inteligencia esta menos no prompt isolado e mais no harness: filesystem, git diff, testes e criterio de parada.

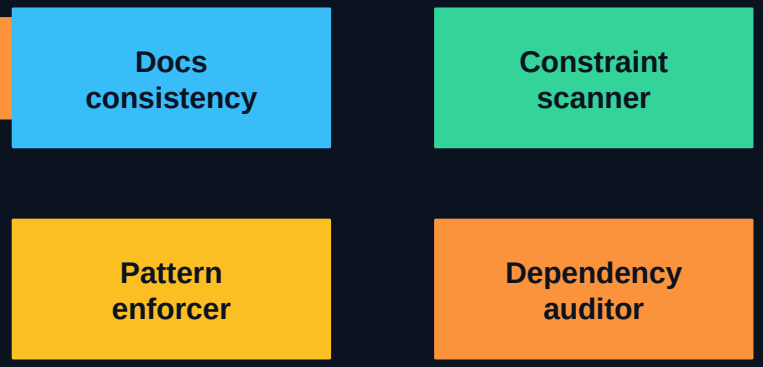
Restricoes arquiteturais e controle de entropia

O Harness deve impor regras mecanicamente e limpar desvios acumulados.

Dependency layering

Cada camada so importa das camadas a esquerda. Validar com lint, testes estruturais, hooks e CI.

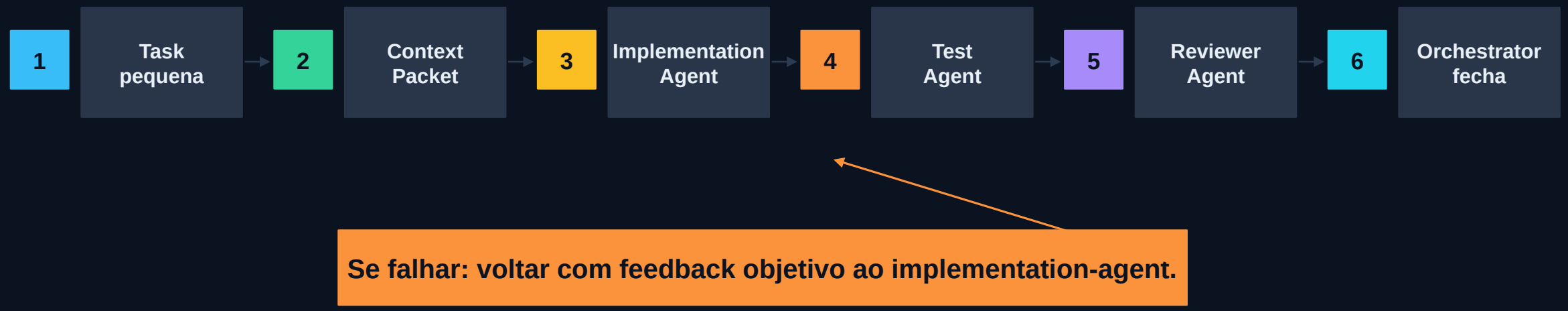
Entropy management



Rodar em PRs, releases ou agendas periodicas para manter o codigo saudavel para humanos e agentes futuros.

Workflow recomendado para Copilot Agents

Um fluxo corporativo precisa de contexto minimo, agentes especializados e evidencia antes de fechar.



Estado confiavel: docs/context-map.md, ADRs, plano versionado, git diff, testes, build, lint e handoff estruturado.

Conclusao

O ganho real vem de processo, verificacao e engenharia de contexto - nao apenas de prompts melhores.

1. Agentes devem ter escopo, contrato e permissao claros.
2. O Orchestrator deve rotear contexto, nao acumular contexto.
3. Contexto deve ser versionado em arquivos e handoffs, nao perdido no chat.
4. **Conclusao exige evidencia: testes, lint, build, criterios e riscos.**
5. Poor self-evaluation e entropia precisam de avaliadores, gates e limpeza continua.

Resumo: transforme o LLM de um gerador confiante em um trabalhador controlado por processo.